

GIÁO TRÌNH KIỂM THỬ XÂM NHẬP ỨNG DỤNG DI ĐỘNG

Mobile Application Penetration Testing • Nền tảng iOS

iOS PENTEST

Toàn tập từ cơ bản đến nâng cao

Kiến trúc bảo mật & mô hình phân quyền iOS, phân tích tĩnh/động, reverse engineering, Frida / Objection / MobSF, bypass jailbreak & SSL pinning, OWASP MASVS/MASTG và các lỗi lưu trữ – mã hóa – cấu hình.

TÁC GIẢ

callmelam2310

Phiên bản 1.0 • 2026

Lời nói đầu

Giáo trình này được biên soạn làm tài liệu học tập và tham chiếu thực hành cho học viên, kỹ sư bảo mật và lập trình viên iOS muốn nắm vững quy trình **kiểm thử xâm nhập ứng dụng di động iOS (iOS Application Penetration Testing)**. Nội dung bám sát khung tham chiếu **OWASP Mobile Application Security (MASVS & MASTG)** và được hệ thống hóa theo một lộ trình từ kiến trúc nền tảng, thiết lập môi trường, cho tới từng kỹ thuật phân tích và khai thác cụ thể.

TUYÊN BỐ SỬ DỤNG HỢP PHÁP

Toàn bộ kỹ thuật trong giáo trình chỉ được phép áp dụng trên **ứng dụng do chính bạn sở hữu**, môi trường lab (DVIA-v2, iGoat, OWASP MASTG apps) hoặc hệ thống bạn có **văn bản ủy quyền kiểm thử (scope) rõ ràng**. Truy cập, dò xét hay can thiệp vào ứng dụng/thiết bị của người khác khi chưa được phép là hành vi vi phạm pháp luật. Tác giả và tài liệu không chịu trách nhiệm cho việc sử dụng sai mục đích.

Đối tượng & kiến thức tiên quyết

- Hiểu cơ bản về dòng lệnh Linux/macOS, mạng TCP/IP và HTTP(S).
- Đã từng làm quen với một ngôn ngữ lập trình (Swift/Objective-C, Python hoặc JavaScript là lợi thế).
- Một máy Mac (khuyến nghị) hoặc Windows/Linux, và một thiết bị iOS đã jailbreak để thực hành.

Bộ công cụ chính sử dụng xuyên suốt

Nhóm	Công cụ tiêu biểu
Thiết bị & quản lý	3uTools, ideviceinstaller, ios-deploy, Apple Configurator
Jailbreak & repo	checkra1n / palera1n, Cydia / Sileo, Frida repo, AppSync
Phân tích tĩnh	Hopper Disassembler, Ghidra, class-dump, otool, nm, jtool2, MobSF
Phân tích động	Frida, Objection, cycrypt, lldb, debugserver
Mạng & traffic	Burp Suite, mitmproxy, Wireshark, tcpdump
Trích xuất & file	frida-ios-dump, plutil, sqlite3, keychain_dumper

CÁCH ĐỌC GIÁO TRÌNH

Mỗi chương đều có phần lý thuyết ngắn gọn, lệnh/PoC minh họa trong khối mã màu, và ô ghi chú (GHI CHÚ / CẢNH BÁO / MẸO). Bạn nên dùng lab riêng và thực hành lại từng lệnh để ghi nhớ.

Mục lục

Lời nói đầu	2
Mục lục	3
Kiến trúc bảo mật & mô hình phân quyền iOS	6
1.1 Chuỗi tin cậy khi khởi động (Secure Boot Chain)	6
1.2 Các lớp bảo mật cốt lõi	6
1.3 Mô hình phân quyền (Permission Model)	6
Cấu trúc ứng dụng iOS & các thành phần	8
2.1 Cấu trúc gói .ipa / .app	8
2.2 Các thư mục dữ liệu trên thiết bị	8
2.3 Các thành phần & cơ chế	8
Thiết lập môi trường pentest iOS	9
3.1 Chuẩn bị thiết bị & jailbreak	9
3.2 3uTools — quản lý thiết bị	9
3.3 Cydia / Sileo: repo & tweaks	9
3.4 Kết nối & port-forward	9
Phân tích tĩnh & phân tích động	11
4.1 Phân tích tĩnh (Static Analysis)	11
4.2 Phân tích động (Dynamic Analysis)	11
Reverse Engineering với Xcode & Hopper	12
5.1 Hopper Disassembler	12
5.2 Xcode / LLDB & debugserver	12
Phân tích lưu lượng: Burp Suite & Wireshark	13
6.1 Cấu hình Burp Suite làm proxy HTTPS	13
6.2 Wireshark & tcpdump	13
Frida, Objection & MobSF	14
7.1 Frida	14

7.2 Objection	14
7.3 MobSF (Mobile Security Framework)	14
Bypass phát hiện jailbreak & SSL Pinning	15
8.1 Cơ chế phát hiện jailbreak	15
8.2 Bypass phát hiện jailbreak	15
8.3 SSL Pinning & cách vượt	15
Local Authentication & bypass Touch ID / Face ID	16
9.1 Điểm yếu thiết kế	16
9.2 Bypass bằng Frida / Objection	16
Rò rỉ dữ liệu, logging, xử lý lỗi & bộ nhớ	17
10.1 Logging quá mức / không an toàn	17
10.2 Rò rỉ dữ liệu qua kênh phụ (Side Channel)	17
10.3 Xử lý lỗi & ngoại lệ (Error & Exception Handling)	17
10.4 Vấn đề quản lý bộ nhớ (Memory Management)	17
WebView, hardcoded, obfuscation, cấu hình backend & binary	18
11.1 Vấn đề WebView	18
11.2 Hardcode bí mật (Hardcoding Issues)	18
11.3 Obfuscation trong mã	18
11.4 Cấu hình sai Firebase & Appshot	18
11.5 Kiểm tra chữ ký binary (Binary Signing)	18
11.6 Phân tích file .plist	18
Lưu trữ, mã hóa, Keychain, quyền & thực hành code an toàn	20
12.1 Lưu trữ dữ liệu không an toàn (Insecure Data Storage)	20
12.2 Băm & mã hóa yếu (Weak Hashing / Crypto)	20
12.3 Keychain dump	20
12.4 Quyền không an toàn (Insecure Permissions)	20
12.5 Thành phần lỗi thời & có lỗi hỏng (Outdated Components)	21
12.6 Thực hành lập trình không an toàn (Insecure Coding)	21
Best practices, checklist & báo cáo	22
13.1 Cấu hình sai phổ biến & best practices	22

13.2 Checklist kiểm thử iOS (rút gọn)	22
13.3 Viết báo cáo	22

NỀN TẢNG

CHƯƠNG 01

Kiến trúc bảo mật & mô hình phân quyền iOS

iOS được thiết kế theo triết lý “secure by default” với nhiều lớp phòng thủ chồng lên nhau. Hiểu rõ các lớp này giúp bạn biết một lỗ hổng nằm ở đâu và vì sao một kỹ thuật khai thác lại khả thi hoặc bị chặn.

1.1 Chuỗi tin cậy khi khởi động (Secure Boot Chain)

Mỗi giai đoạn khởi động đều được ký số và xác minh bởi giai đoạn trước đó: **Boot ROM** (gốc tin cậy bất biến) → **iBoot** → **Kernel**. Nếu bất kỳ chữ ký nào không hợp lệ, thiết bị vào chế độ phục hồi. Đây là lý do jailbreak thường dựa vào lỗ hổng tầng thấp (ví dụ checkm8 trên Boot ROM của A5–A11).

1.2 Các lớp bảo mật cốt lõi

Secure Enclave (SEP)	Bộ đồng xử lý cách ly, quản lý khóa mã hóa, Touch ID/Face ID; dù iOS bị khai thác SEP vẫn tách biệt.
Data Protection	Mã hóa file theo từng lớp (NSFileProtection); khóa được dẫn xuất từ passcode + UID phần cứng.
Code Signing	Mọi đoạn mã thực thi phải có chữ ký hợp lệ; chặn chạy mã tùy ý (mandatory code signing).
Sandbox (Seatbelt)	Mỗi app chạy trong container riêng, không đọc/ghi được vùng của app khác.
ASLR + DEP/NX	Ngẫu nhiên hóa địa chỉ và không cho thực thi vùng dữ liệu, chống khai thác bộ nhớ.
Entitlements	Quyền đặc biệt gắn vào chữ ký app (ví dụ keychain-access-groups, app groups).

1.3 Mô hình phân quyền (Permission Model)

Người dùng cấp quyền tại thời điểm chạy (runtime) cho từng tài nguyên nhạy cảm. Nhà phát triển phải khai báo mục đích sử dụng trong **Info.plist** qua các khóa `Usage Description`; thiếu khóa này app sẽ bị crash khi truy cập tài nguyên.

Tài nguyên	Khóa Info.plist tiêu biểu
Camera	<code>NSCameraUsageDescription</code>
Vị trí	<code>NSLocationWhenInUseUsageDescription</code>
Danh bạ	<code>NSContactsUsageDescription</code>
Ảnh / Thư viện	<code>NSPhotoLibraryUsageDescription</code>
Microphone	<code>NSMicrophoneUsageDescription</code>

GÓC NHÌN PENTESTER

Khi review app, kiểm tra app có xin quyền **vượt quá nhu cầu chức năng** không (over-privileged) và các khóa Usage Description có bị dùng để “xã hội kỹ thuật” người dùng không. Xem chi tiết ở Chương 12 (Insecure Permissions).

NỀN TẢNG

CHƯƠNG 02

Cấu trúc ứng dụng iOS & các thành phần

Trước khi tấn công, bạn phải biết một gói .ipa gồm những gì, dữ liệu nằm ở đâu trên đĩa và vòng đời của app được điều khiển ra sao.

2.1 Cấu trúc gói .ipa / .app

File .ipa thực chất là một file ZIP. Giải nén sẽ thấy:

```
Payload/
  MyApp.app/
    MyApp                <- Mach-O binary (thuong da ma hoa FairPlay)
    Info.plist           <- metadata, bundle id, permissions, URL schemes
    embedded.mobileprovision <- provisioning profile + entitlements
    _CodeSignature/     <- chu ky so
    Assets.car, *.nib, *.storyboardc, Frameworks/, PlugIns/
```

2.2 Các thư mục dữ liệu trên thiết bị

Bundle container	/private/var/containers/Bundle/Application/<UUID>/ – chứa .app, chỉ đọc.
Data container	/private/var/mobile/Containers/Data/Application/<UUID>/ – Documents, Library, tmp.
Documents/	Dữ liệu người dùng, được backup iCloud/iTunes.
Library/Caches/	Cache, thường bị bỏ qua khi backup nhưng vẫn nằm trên đĩa.
Library/Preferences/	File .plist lưu UserDefaults.
tmp/	Dữ liệu tạm, hệ thống có thể xóa.

2.3 Các thành phần & cơ chế

- **URL Schemes / Universal Links / App Links** — cổng deep-link, thường là bề mặt tấn công (IPC liên app).
- **App Extensions** (Share, Today, Keyboard) — chạy trong tiến trình riêng, có thể rò rỉ dữ liệu.
- **Background modes, Push notification, Pasteboard** chung — kênh rò rỉ dữ liệu tiềm ẩn.
- **NSUserDefaults, Keychain, Core Data, SQLite, Realm** — các kho lưu trữ cần kiểm tra (Chương 12).

MẸO

Đọc Info.plist để liệt kê nhanh URL schemes (CFBundleURLTypes) — đây thường là điểm vào đầu tiên khi test IPC.

MÔI TRƯỜNG

CHƯƠNG 03

Thiết lập môi trường pentest iOS

Một môi trường ổn định giúp bạn lặp lại kết quả. Chương này hướng dẫn chuẩn bị thiết bị, jailbreak, kho Cydia và các công cụ hỗ trợ.

3.1 Chuẩn bị thiết bị & jailbreak

Nhiều kỹ thuật phân tích động đòi hỏi thiết bị jailbreak để bỏ qua sandbox và gắn Frida. Lựa chọn công cụ jailbreak tùy dòng chip & phiên bản iOS:

Công cụ	Kiểu	Phạm vi tiêu biểu
checkra1n / palera1n	Semi-tethered (BootROM)	Thiết bị A9–A11 (checkm8)
unc0ver / Taurine	Semi-untethered	Một số bản iOS 12–14
Dopamine	Rootless	arm64e, iOS 15–16.x

CẢNH BÁO

Chỉ jailbreak **thiết bị test riêng**. Jailbreak có thể làm mất bảo hành, tăng bề mặt tấn công và không nên dùng trên thiết bị cá nhân chứa dữ liệu thật.

3.2 3uTools — quản lý thiết bị

3uTools là tiện ích Windows tiện lợi để: xem thông tin thiết bị & chứng chỉ, sao lưu/khôi phục, cài .ipa, quản lý file hệ thống, chụp log & ảnh màn hình. Rất hữu ích để **cài app cần test** và **trích xuất file** mà không cần dòng lệnh.

3.3 Cydia / Sileo: repo & tweaks

Sau khi jailbreak, cài các gói hỗ trợ pentest qua Cydia/Sileo. Các repo & gói thường dùng:

- **Frida repo** — <https://build.frida.re> → cài gói *Frida*.
- **OpenSSH** — truy cập SSH vào thiết bị (mặc định đổi mật khẩu alpine ngay).
- **AppSync Unified** — cho phép cài app đã ký lại (self-signed).
- **Tweaks kiểm thử**: SSL Kill Switch 3, FlyJB/Liberty Lite (test jailbreak detection), Filza (file manager).

ĐỔI MẬT KHẨU SSH NGAY

Sau khi cài OpenSSH, đổi ngay mật khẩu mặc định alpine cho cả user root và mobile để tránh bị chiếm thiết bị qua Wi-Fi.

3.4 Kết nối & port-forward

```
# Ket noi SSH qua USB (iproxy tu libimobiledevice)
iproxy 2222 22 &
ssh root@127.0.0.1 -p 2222      # mat khau mac dinh: alpine (hay doi!)

# Cai .ipa qua dong lenh
ideviceinstaller -i MyApp.ipa
```

PHÂN TÍCH

CHƯƠNG 04

Phân tích tĩnh & phân tích động

Hai trụ cột của pentest ứng dụng: phân tích tĩnh (đọc binary, cấu hình, mã nguồn khi không chạy) và phân tích động (quan sát và can thiệp app khi đang chạy).

4.1 Phân tích tĩnh (Static Analysis)

- Trích .ipa, đọc Info.plist, entitlements và provisioning profile.
- Kiểm tra cơ chế bảo vệ binary: PIE/ASLR, stack canary, ARC, encryption (Chương 11).
- class-dump / Hopper để lấy header, tên lớp & hàm Objective-C; tìm chuỗi nhạy cảm (hardcoded secrets).
- Grep chuỗi: API key, URL, endpoint, mật khẩu, comment để lộ.

```
# Xem kien truc & co che bao ve
otool -hv MyApp.app/MyApp          # PIE flag
otool -Iv MyApp | grep stack_chk   # stack canary
otool -L MyApp                      # thu vien lien ket

# Liet ke lop/ham Objective-C
class-dump -H MyApp.app/MyApp -o headers/

# San chuoi nhạy cam
strings -a MyApp | egrep -i 'api|key|secret|token|http|password'
```

4.2 Phân tích động (Dynamic Analysis)

- Quan sát file/log/keychain thay đổi khi thao tác app (runtime storage).
- Hook hàm bằng Frida/Objection để đọc/sửa tham số, giá trị trả về.
- Chặn & sửa traffic mạng (Chương 6); theo dõi lệnh SQL, crypto, IPC.
- Debug bằng lldb + debugserver để đặt breakpoint, đọc thanh ghi/bộ nhớ.

KẾT HỢP HAI CHIỀU

Dùng phân tích tĩnh để **tìm mục tiêu** (tên lớp/hàm, offset), rồi phân tích động để **xác nhận & khai thác**. Vòng lặp tĩnh ↔ động này là trái tim của reverse engineering ứng dụng.

PHÂN TÍCH

CHƯƠNG 05

Reverse Engineering với Xcode & Hopper

Dịch ngược binary Mach-O giúp hiểu logic bên trong: thuật toán, kiểm tra bảo mật phía client, chỗ giấu bí mật. Xcode/LLDB mạnh cho debug động, Hopper/Ghidra mạnh cho đọc pseudo-code tĩnh.

5.1 Hopper Disassembler

- Nạp binary đã **giải mã FairPlay** (xem Chương 11) để đọc được mã thật.
- Chuyển giữa chế độ ASM / CFG / pseudo-code (*Show Pseudo Code*) để đọc dễ hơn.
- Tìm hàm theo tên (ví dụ `isJailbroken`, `validatePinning`), phân tích luồng điều kiện.
- Đánh dấu & sửa nhãn, xuất ghi chú; kết hợp với class-dump để đối chiếu selector Objective-C.

5.2 Xcode / LLDB & debugserver

```
# Tren thiết bị jailbreak: khởi chạy debugserver
debugserver *:1234 -a MyApp &
# Tren máy tính: gõ lldb
lldb
(lldb) process connect connect://127.0.0.1:1234
(lldb) breakpoint set -n "-[LoginVC validate:]"
(lldb) register read
(lldb) memory read --size 1 --format x $x0
```

GHI CHÚ

Xcode chủ yếu mạnh khi bạn có thể build lại/attach vào app do mình sở hữu. Với app third-party, LLDB qua debugserver + Hopper là bộ đôi thực tế hơn. Ghidra là lựa chọn miễn phí thay thế Hopper.

MẠNG

CHƯƠNG 06

Phân tích lưu lượng: Burp Suite & Wireshark

Đa số dữ liệu nhạy cảm đi qua mạng. Chặn và phân tích HTTP(S) bằng proxy, và soi gói ở tầng thấp bằng Wireshark khi giao thức không phải HTTP.

6.1 Cấu hình Burp Suite làm proxy HTTPS

- Đặt Burp lắng nghe trên IP LAN (Proxy → Options → bind to all interfaces).
- Trên iPhone: Settings → Wi-Fi → cấu hình HTTP Proxy trở về IP:port của Burp.
- Cài chứng chỉ CA của Burp: tải qua `http://burp`, rồi **bật Trust toàn phần** trong Settings → General → About → Certificate Trust Settings.
- Nếu app dùng **SSL pinning**, chứng chỉ Burp sẽ bị từ chối → cần bypass (Chương 8).

iOS ATS

iOS áp App Transport Security buộc HTTPS & TLS hiện đại. Nếu thấy app cho phép HTTP thường hoặc hạ thấp ATS trong Info.plist (NSAllowsArbitraryLoads), đó là một phát hiện cần báo cáo.

6.2 Wireshark & tcpdump

Với giao thức không phải HTTP (WebSocket nhị phân, MQTT, giao thức riêng), Burp không đủ. Dùng tcpdump trên thiết bị hoặc tạo Remote Virtual Interface (RVI) trên macOS để bắt gói bằng Wireshark:

```
# macOS: tạo interface ảo gắn với UDID thiết bị
rvtctl -s <UDID>
tcpdump -i rvi0 -w capture.pcap
# Sau đó mở capture.pcap trong Wireshark để phân tích
```

PHÂN TÍCH

CHƯƠNG 07

Frida, Objection & MobSF

Bộ ba công cụ chủ lực: Frida (instrumentation động linh hoạt), Objection (khung dựa trên Frida có sẵn nhiều lệnh), và MobSF (nền tảng phân tích tự động cả tĩnh lẫn động).

7.1 Frida

```
frida-ps -Ua # liệt kê app đang chạy trên thiết bị USB
frida-trace -U -n MyApp -m "-[* *password*]" # trace selector chứa 'password'

// hook.js - ví dụ hook một hàm Objective-C
if (ObjC.available) {
  var cls = ObjC.classes.LoginManager;
  Interceptor.attach(cls['- validatePassword:'].implementation, {
    onEnter: function (args) {
      console.log('pwd =', ObjC.Object(args[2]).toString());
    },
    onLeave: function (retval) { retval.replace(0x1); } // ép trả về true
  });
}
```

7.2 Objection

```
objection -g MyApp explore
ios plist cat Info.plist
ios keychain dump
ios nsuserdefaults get
ios cookies get
ios sslpinning disable
ios jailbreak disable
memory search --string "token"
```

7.3 MobSF (Mobile Security Framework)

- Tải .ipa lên MobSF để tự động: đọc Info.plist/entitlements, chấm điểm cấu hình, phát hiện hardcoded secrets, kiểm tra binary protections, và phát sinh báo cáo.
- MobSF hỗ trợ cả **dynamic analysis** khi kết nối thiết bị jailbreak (qua Frida), giúp quan sát file/log runtime.
- Dùng MobSF để **quét nhanh** tạo baseline, sau đó điều tra thủ công những điểm nghi ngờ.

QUY TRÌNH

MobSF (quét nhanh) → xác nhận bằng Frida/Objection → dịch ngược sâu bằng Hopper. Tự động hóa để phủ rộng, thủ công để đào sâu.

KỸ THUẬT

CHƯƠNG 08

Bypass phát hiện jailbreak & SSL Pinning

Nhiều app tự bảo vệ bằng cách từ chối chạy trên thiết bị jailbreak hoặc ghim chứng chỉ (SSL pinning). Pentester phải vượt qua các lớp này để phân tích, đồng thời đánh giá độ vững chắc của chúng.

8.1 Cơ chế phát hiện jailbreak

- Kiểm tra sự tồn tại file/đường dẫn: /Applications/Cydia.app, /bin/bash, /etc/apt.
- Thử ghi ra ngoài sandbox (ví dụ /private/jb.txt).
- Kiểm tra URL scheme cydia://; gọi fork(); kiểm tra dynamic libraries bất thường.

8.2 Bypass phát hiện jailbreak

```
# Objection - tắt các kiểm tra phổ biến
ios jailbreak disable

# Frida - ví dụ giả mạo fileExistsAtPath: cho các đường dẫn lo
var NSFileManager = ObjC.classes.NSFileManager;
var sel = NSFileManager['- fileExistsAtPath:'];
Interceptor.attach(sel.implementation, {
  onEnter: function(a){ this.p = ObjC.Object(a[2]).toString(); },
  onLeave: function(r){ if(/Cydia|apt|bash/.test(this.p)) r.replace(0x0); }
});
```

8.3 SSL Pinning & cách vượt

SSL pinning gắn cứng chứng chỉ/ khóa công khai mong đợi, khiến proxy như Burp bị từ chối. Các hướng bypass:

- **Objection:** ios sslpinning disable (hook các API TrustKit/NSURLSession phổ biến).
- **Frida script** (ví dụ bộ script pinning cộng đồng) hook SecTrustEvaluate / TrustKit.
- **SSL Kill Switch 3** (tweak Cydia) tắt pinning ở tầng hệ thống.
- Nếu app dùng thư viện tùy chỉnh, đọc ngược bằng Hopper để tìm hàm xác minh và hook riêng.

ĐÁNH GIÁ CHỨ KHÔNG CHỈ VƯỢT

Bypass thành công không phải là kết luận cuối. Hãy báo cáo **mức độ khó**: pinning có bị vượt chỉ bằng công cụ sẵn không? Có kết hợp chống hook/anti-Frida không? Đó là thông tin giá trị cho khách hàng.

KỸ THUẬT

CHƯƠNG 09

Local Authentication & bypass Touch ID / Face ID

Xác thực sinh trắc (LocalAuthentication framework) nếu chỉ dùng ở tầng UI mà không gắn với bảo vệ dữ liệu thật trong Keychain thì có thể bị vượt bằng instrumentation.

9.1 Điểm yếu thiết kế

- App chỉ gọi `evaluatePolicy` và tin vào kết quả callback (kiểm tra phía client).
- Không ràng buộc dữ liệu nhạy cảm vào `SecAccessControl` với cờ `.biometryCurrentSet/.userPresence`.
- Logic “thành công” nằm trong ứng dụng thay vì để Secure Enclave giải phóng khóa.

9.2 Bypass bằng Frida / Objection

```
# Objection: ep evaluatePolicy tra ve thanh cong
ios ui biometrics_bypass

# Frida: hook LAContext evaluatePolicy:localizedReason:reply:
var LAContext = ObjC.classes.LAContext;
Interceptor.attach(LAContext['- evaluatePolicy:localizedReason:reply:'].implementation, {
  onEnter: function (args) {
    var block = new ObjC.Block(args[4]);
    var orig = block.implementation;
    block.implementation = function (ok, err) { orig(true, null); };
  }
});
```

CÁCH LÀM ĐÚNG

Khuyến nghị cho dev: gắn khóa/dữ liệu vào Keychain với `SecAccessControlCreateWithFlags(..., .biometryCurrentSet, ...)` để việc vượt UI không thể mở khóa được dữ liệu.

LỖ HỔNG

CHƯƠNG 10

Rò rỉ dữ liệu, logging, xử lý lỗi & bộ nhớ

Nhóm các lỗi thường gặp liên quan đến việc dữ liệu nhạy cảm “bị lộ ra ngoài” mà app không nhận ra: log quá mức, kênh phụ (side channel), thông báo lỗi lộ thông tin và vấn đề quản lý bộ nhớ.

10.1 Logging quá mức / không an toàn

- Ghi token, mật khẩu, dữ liệu cá nhân ra NSLog/os_log, xem qua Console.app hoặc idevicesyslog.
- Bản debug còn bật logging chi tiết khi phát hành production.

```
idevicesyslog | egrep -i 'token|password|jwt|card|otp' # theo dõi log hệ thống
```

10.2 Rò rỉ dữ liệu qua kênh phụ (Side Channel)

- **Application snapshot:** iOS chụp màn hình khi app vào nền → Library/Caches/Snapshots/ có thể lộ dữ liệu nhạy cảm (xem thêm Chương 11).
- **Pasteboard** chung: dữ liệu copy bị app khác đọc.
- **Keyboard cache:** text field không tắt autocorrect có thể lưu dữ liệu nhạy cảm.
- **Cookies, WebKit cache, crash log** chứa thông tin phiên/dữ liệu.

10.3 Xử lý lỗi & ngoại lệ (Error & Exception Handling)

Thông báo lỗi quá chi tiết (stack trace, câu truy vấn SQL, đường dẫn hệ thống, phiên bản thư viện) giúp kẻ tấn công hiểu hệ thống. Kiểm tra cả lỗi phía client hiển thị cho người dùng lẫn phản hồi lỗi từ API.

10.4 Vấn đề quản lý bộ nhớ (Memory Management)

- Dữ liệu nhạy cảm (mật khẩu, khóa) tồn tại lâu trong bộ nhớ, không được xóa → có thể đọc bằng memory dump (Objection: memory search).
- Lỗi quản lý con trỏ trong mã C/Objective-C (use-after-free, buffer overflow) — hiếm hơn nhưng nghiêm trọng.

CẢNH BÁO

Kiểm tra bộ nhớ có thể lộ mật khẩu/token ngay cả khi không lưu trên đĩa. Khuyến nghị dev dùng kiểu dữ liệu có thể “xóa an toàn” và giảm thời gian dữ liệu nhạy cảm tồn tại trong RAM.

LỖ HỔNG

CHƯƠNG 11

WebView, hardcode, obfuscation, cấu hình backend & binary

Nhóm các lỗi ở tầng triển khai: WebView bất cẩn, bí mật nhúng trong mã, thiếu (hoặc lạm dụng) obfuscation, cấu hình đám mây sai, appshot và các vấn đề ký/chữ ký binary, phân tích .plist.

11.1 Vấn đề WebView

- Dùng UIWebView (đã khai tử) thay vì WKWebView.
- Bật `JavaScriptEnabled` + nạp nội dung không tin cậy → nguy cơ XSS/inject.
- Cầu JavaScript bridge (`messageHandlers`) lộ API nhạy cảm ra JS.
- Nạp file cục bộ bằng `file://` không kiểm soát → local file inclusion.

11.2 Hardcode bí mật (Hardcoding Issues)

API key, secret, chuỗi kết nối, khóa mã hóa nhúng cứng trong binary/plist đều có thể trích xuất dễ dàng. Chúng không được coi là bí mật.

```
strings -a MyApp | egrep -i 'AKIA|apikey|secret|-----BEGIN'
plutil -p Info.plist | egrep -i 'key|token|url'
```

11.3 Obfuscation trong mã

Obfuscation làm khó reverse engineering nhưng **không thay thế** các biện pháp bảo mật thật. Đánh giá: app có obfuscate tên lớp/chuỗi/luồng điều khiển không; obfuscation có bị tháo gỡ dễ bằng công cụ tự động không. Thiếu obfuscation cho phần bảo vệ quan trọng (pinning, license) là một ghi nhận.

11.4 Cấu hình sai Firebase & Appshot

- **Firebase Realtime DB mở:** thử truy cập `https://<project>.firebaseio.com/.json` — nếu trả về dữ liệu không cần xác thực là cấu hình rule sai.
- Lấy `GoogleService-Info.plist` trong bundle để biết project id/endpoint.
- **Appshot** (snapshot khi vào nền): kiểm tra app có che (blur/ảnh trống) màn hình nhạy cảm trước khi vào nền không.

11.5 Kiểm tra chữ ký binary (Binary Signing)

```
codesign -dvvv MyApp.app # xem chi tiết chu ký
codesign --entitlements :- MyApp.app # xem entitlements
otool -l MyApp | grep -A5 LC_ENCRYPTION # cryptid: 1 = con ma hoa FairPlay
```

11.6 Phân tích file .plist

.plist có thể ở dạng XML hoặc nhị phân. Dùng `plutil` để đọc và chuyển đổi. Kiểm tra `Info.plist` (permissions, URL schemes, ATS), file Preferences (`NSUserDefaults`) tìm dữ liệu nhạy cảm lưu dạng rõ.

```
plutil -convert xml1 -o - Library/Preferences/com.app.plist  
plutil -p Root.plist
```

LỖ HỔNG

CHƯƠNG 12

Lưu trữ, mã hóa, Keychain, quyền & thực hành code an toàn

Chương tổng hợp các lỗi cốt lõi trong MASVS: lưu trữ không an toàn, thuật toán băm/mã hóa yếu, dump Keychain, quyền sai, dùng thành phần lỗi thời và thói quen lập trình không an toàn.

12.1 Lưu trữ dữ liệu không an toàn (Insecure Data Storage)

- Dữ liệu nhạy cảm trong **NSUserDefaults** (plist rõ), **SQLite/Core Data/Realm** không mã hóa, hay file trong Documents/Caches.
- Kiểm tra bằng cách thao tác app rồi soi lại container dữ liệu (Chương 2) hoặc dump qua Objection.

```
# Doc SQLite trích tu container
sqlite3 Documents/app.db '.tables'
sqlite3 Documents/app.db 'SELECT * FROM users;'
```

12.2 Băm & mã hóa yếu (Weak Hashing / Crypto)

Vấn đề	Ví dụ cần cảnh giác	Khuyến nghị
Hash yếu	MD5, SHA-1 cho mật khẩu	SHA-256+, hoặc bcrypt/scrypt/Argon2 cho pass
Chế độ yếu	AES-ECB, IV cố định/đoán được	AES-GCM, IV ngẫu nhiên
Khóa cứng	Khóa nhúng trong binary	Dẫn xuất qua Keychain/SEP
Random yếu	rand()/arc4random đoán được	SecRandomCopyBytes

12.3 Keychain dump

Keychain là kho lưu khóa/mật khẩu của iOS. Trên thiết bị jailbreak có thể dump để kiểm tra app có lưu bí mật sai cách hay đặt thuộc tính truy cập quá rộng (ví dụ kSecAttrAccessibleAlways).

```
objection -g MyApp explore
ios keychain dump
# Chu ý thuộc tính accessibility: WhenUnlocked / AfterFirstUnlock / Always
```

THUỘC TÍNH ACCESSIBILITY

kSecAttrAccessibleAlways và ...AlwaysThisDeviceOnly (đã khai tử) cho phép đọc khi thiết bị khóa — nên dùng WhenUnlockedThisDeviceOnly cho dữ liệu nhạy cảm.

12.4 Quyền không an toàn (Insecure Permissions)

- App xin nhiều quyền hơn nhu cầu (camera, danh bạ, vị trí nền...).
- URL scheme/Universal Link nhận tham số tin cậy mà không xác minh nguồn gốc → IPC bị lạm dụng.
- App Group/keychain-access-group chia sẻ quá rộng giữa các app.

12.5 Thành phần lỗi thời & có lỗ hổng (Outdated Components)

- Thư viện/SDK (Alamofire, AFNetworking, OpenSSL nhúng...) phiên bản cũ có CVE đã công bố.
- Đối chiếu Frameworks/ và Pods/Package.resolved với cơ sở dữ liệu CVE.
- UIWebView, TLS cũ, thuật toán đã khai tử đều là cờ đỏ.

12.6 Thực hành lập trình không an toàn (Insecure Coding)

- Tin tưởng dữ liệu đầu vào không kiểm chứng (deep link, response API, WebView message).
- Format string, SQL nối chuỗi (SQL injection cục bộ), đường dẫn file không lọc.
- Logic bảo mật quan trọng đặt ở phía client (có thể vượt bằng Frida) thay vì phía server.

TỔNG KẾT

CHƯƠNG 13

Best practices, checklist & báo cáo

Chương cuối tổng hợp các cấu hình sai phổ biến, best practice bảo mật iOS và một checklist thực hành để bạn tự kiểm soát công việc, cùng gợi ý cách viết báo cáo.

13.1 Cấu hình sai phổ biến & best practices

Hạng mục	Best practice
Truyền tải	Buộc TLS (ATS bật), pinning đúng, không hạ thấp ATS toàn cục
Lưu trữ	Bí mật vào Keychain (accessibility chặt), không lưu rõ trong UserDefaults/SQLite
Mã hóa	AES-GCM + IV ngẫu nhiên; hash pass bằng Argon2/bcrypt; khóa qua SEP
Xác thực cục bộ	Ràng buộc sinh trắc với SecAccessControl (.biometryCurrentSet)
Logging	Tắt log nhạy cảm ở production; không ghi PII/token
WebView	WKWebView, tắt JS nếu không cần, kiểm soát bridge & origin
Bảo vệ client	Jailbreak detection + anti-hook là lớp hỗ trợ, không thay bảo mật server
Backend	Firebase rules chặt; không hardcode secret; xác thực phía server

13.2 Checklist kiểm thử iOS (rút gọn)

- Thu thập: .ipa, Info.plist, entitlements, Frameworks, URL schemes.
- Binary: PIE, stack canary, ARC, cryptid, chữ ký & entitlements.
- Lưu trữ: UserDefaults, Keychain dump, SQLite/Realm/Core Data, file container.
- Mạng: chặn Burp, kiểm tra pinning, ATS, dữ liệu nhạy cảm trên đường truyền.
- Runtime: Frida/Objection hook auth, jailbreak detection, biometrics.
- Rò rỉ: log hệ thống, snapshot/appshot, pasteboard, keyboard cache, crash log.
- Crypto: thuật toán/chế độ/khóa; random; hardcoded secrets.
- Backend: Firebase mở, endpoint lộ, thành phần lỗi thời (CVE).

13.3 Viết báo cáo

- Mỗi phát hiện: mô tả, mức độ (CVSS/rủi ro), bước tái hiện (PoC), ảnh chứng minh, khuyến nghị khắc phục.
- Tham chiếu MASVS/MASTG để chuẩn hóa; sắp xếp theo độ ưu tiên khắc phục.
- Phân biệt rõ **rủi ro thực tế** và **ghi nhận củng cố** (hardening) để khách hàng ưu tiên đúng.

Tài liệu tham khảo

- OWASP Mobile Application Security — MASVS & MASTG (owasp.org).
- Apple Platform Security Guide.
- Frida & Objection documentation; MobSF documentation.

— Hết —

Biên soạn bởi callmelam2310 • Phiên bản 1.0 • 2026